

Relatório de Forense

- Captura Criminosa

O arquivo fornecido no chall é de formato pcapng, dessa forma, foi possível abri-lo no programa Wireshark, que faz a análise da troca de pacotes em rede. Explorando o arquivo questão, destacaram-se as comunicações feitas em HTTP, que, diferente da HTTPS, não possui a segurança e está suscetível a interceptação. Foi possível verificar que houve a requisição e o envio de uma imagem png, chamada de “evidencias_do_crime.png”, como mostra a imagem a seguir.

No.	Time	Source	Destination	Protocol	Length	Info
10	0.000000	149.154.175.55	192.168.0.35	SSL	175	Continuation Data
2	0.047083	192.168.0.35	149.154.175.55	TCP	54	63813 → 443 [ACK] Seq=1 Ack=122 Win=2043 Len=0
3	0.269719	2804:14d:5c5a:919e::...	2800:3f0:4004:805::...	TCP	75	53319 → 443 [ACK] Seq=1 Ack=1 Win=252 Len=1
4	0.284462	2800:3f0:4004:805::...	2804:14d:5c5a:919e::...	TCP	86	443 → 53319 [ACK] Seq=1 Ack=2 Win=1046 Len=0 SLE=1 SRE=2
5	0.489788	2801:0:2b77:7aa:18b::...	ff02::1	IPv6	82	IPv6 no next header
6	0.848776	149.154.175.55	192.168.0.35	SSL	159	Continuation Data
7	0.891878	192.168.0.35	149.154.175.55	TCP	54	63813 → 443 [ACK] Seq=1 Ack=227 Win=2047 Len=0
8	2.459821	192.168.0.160	255.255.255.255	UDP	246	59727 → 6667 Len=204
9	2.867750	192.168.0.122	192.168.0.35	TCP	66	40738 → 8000 [FIN, ACK] Seq=1 Ack=1 Win=128 Len=0 TSval=1630284074 TSecr=541965860
10	2.867750	192.168.0.122	192.168.0.35	TCP	74	43258 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=1630284076 TSecr=0 WS=512
11	2.868885	192.168.0.35	192.168.0.122	TCP	74	8000 → 43258 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=542101717 TSecr=1630284074
12	2.868109	192.168.0.35	192.168.0.122	TCP	66	8000 → 40738 [ACK] Seq=1 Ack=2 Win=255 Len=0 TSval=542101717 TSecr=1630284074
13	2.868180	192.168.0.35	192.168.0.122	TCP	66	8000 → 40738 [FIN, ACK] Seq=1 Ack=2 Win=255 Len=0 TSval=542101717 TSecr=1630284074
14	2.883764	192.168.0.122	192.168.0.35	TCP	66	40738 → 8000 [ACK] Seq=2 Ack=2 Win=128 Len=0 TSval=1630284094 TSecr=542101717
15	2.883764	192.168.0.122	192.168.0.35	TCP	66	43258 → 8000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1630284094 TSecr=542101717
16	2.901294	192.168.0.122	192.168.0.35	HTTP	577	GET /evidencias_do_crime.png HTTP/1.1
17	2.902670	192.168.0.122	192.168.0.35	TCP	74	43262 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=1630284107 TSecr=0 WS=512
18	2.902941	192.168.0.35	192.168.0.122	TCP	74	8000 → 43262 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=542101752 TSecr=1630284107
19	2.903968	192.168.0.35	192.168.0.122	TCP	253	8000 → 43258 [PSH, ACK] Seq=1 Ack=512 Win=65024 Len=187 TSval=542101753 TSecr=1630284108 [TCP PDU reassd
20	2.903968	192.168.0.35	192.168.0.122	TCP	173	43258 → 8000 [ACK] Seq=512 Ack=188 Win=67072 Len=0 TSval=1630284128 TSecr=542101753
21	2.906806	192.168.0.35	192.168.0.122	TCP	66	8000 → 43258 [FIN, ACK] Seq=1885 Ack=512 Win=65024 Len=0 TSval=542101754 TSecr=1630284108
22	2.915198	192.168.0.122	192.168.0.35	TCP	66	43258 → 8000 [ACK] Seq=512 Ack=188 Win=67072 Len=0 TSval=1630284128 TSecr=542101753
23	2.916747	192.168.0.122	192.168.0.35	TCP	66	43262 → 8000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1630284128 TSecr=542101752
24	2.916775	192.168.0.122	192.168.0.35	TCP	66	43258 → 8000 [ACK] Seq=512 Ack=188 Win=65024 Len=0 TSval=1630284128 TSecr=542101754
25	2.922461	192.168.0.122	192.168.0.35	TCP	66	43258 → 8000 [FIN, ACK] Seq=512 Ack=1885 Win=72784 Len=0 TSval=1630284130 TSecr=542101754

Transmission Control Protocol, Src Port: 8000, Dst Port: 43258, Seq: 188, Ack: 512, Len: 1

[2 Reassembled TCP Segments (1884 bytes): #19(187), #20(1697)]

HyperText Transfer Protocol

HTTP/1.0 200 OK\r\n

Server: SimpleHTTP/0.6 Python/3.11.9\r\n

Date: Tue, 04 Feb 2025 19:57:03 GMT\r\n

Content-type: image/png\r\n

Content-Length: 1697\r\n

Last-Modified: Tue, 04 Feb 2025 19:47:28 GMT\r\n

\r\n

[Request in frame: 10]

[Time since request: 0.004185000 seconds]

[Request URI: /evidencias_do_crime.png]

[Full request GET http://192.168.0.35:8000/evidencias_do_crime.png]

File Data: 1697 bytes

[Expert Info (Note/Malformed): HTTP body subdissector failed, trying heuristic subdissector]

Media Type

Media type: image/png (1697 bytes)

Media type (media.type), 1697 byte(s)

Frame (1763 bytes) | Reassembled TCP (1884 bytes) | Packets: 43 | Profile: Default

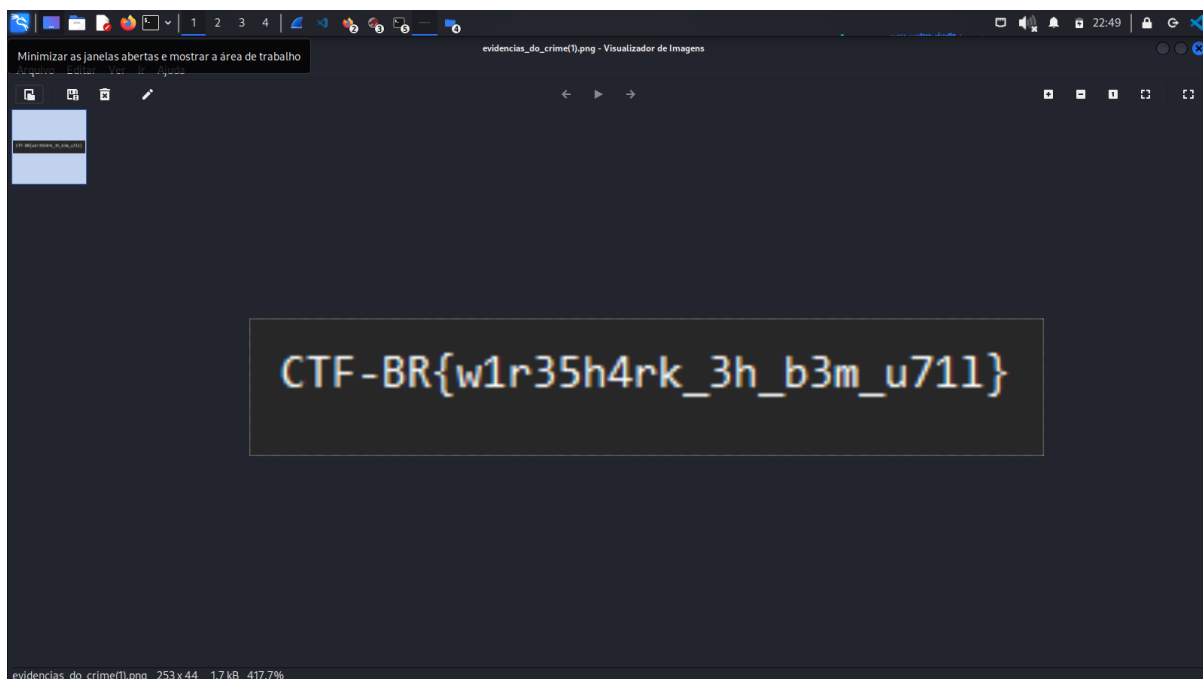
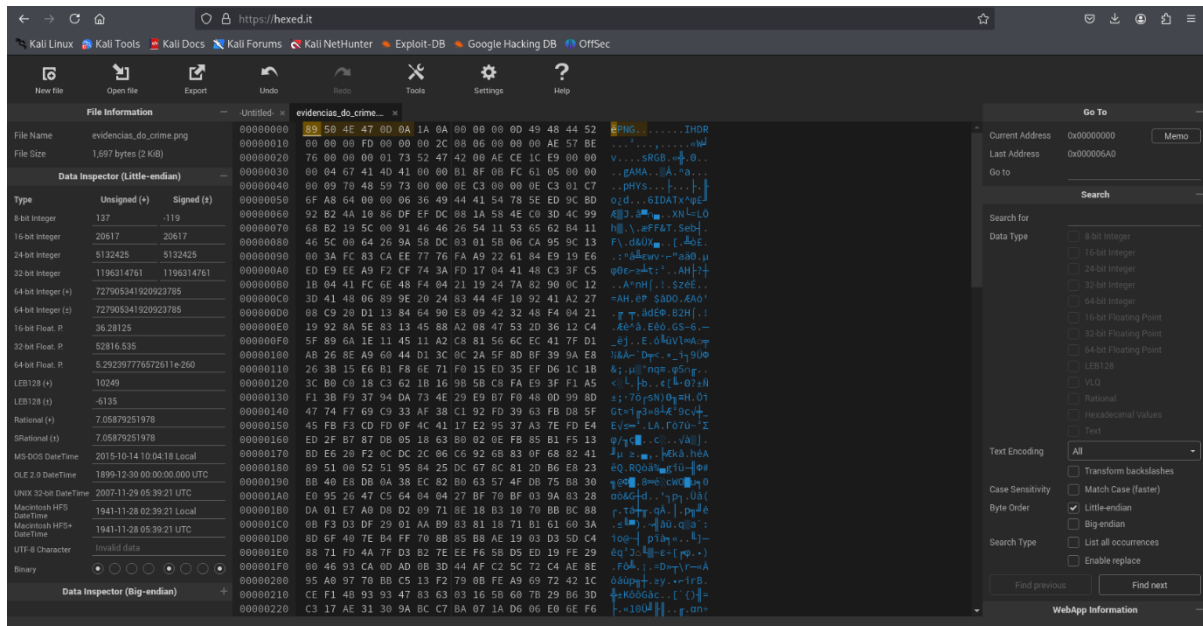
Para analisar melhor a imagem em questão, exportei ela do Wireshark e salvei em meu computador. Ao tentar acessar a imagem, surgiu um erro que afirmou não ser possível ver o arquivo por não ser um png. Verificando o tipo do arquivo e seus magic bytes, de fato se observou que o arquivo não tipo não era do formato png.

```
(gui@kalibon)-[~/Downloads]
$ file evidencias_do_crime.png
evidencias_do_crime.png: data

(gui@kalibon)-[~/Downloads]
$ hexdump -C evidencias_do_crime.png | head
```

Offset	Hex	ASCII	Destination	Protocol
00000000	de ad be ef ca fe 1a 0a 00 00 00 0d 49 48 44 52	IHDR		
00000010	00 00 00 fd 00 00 00 2c 08 06 00 00 00 ae 57 be	W.		
00000020	76 00 00 00 01 73 52 47 42 00 ae ce 1c e9 00 00	v....sRGB.....		
00000030	00 04 67 41 4d 41 00 00 b1 8f 0b fc 61 05 00 00	..gAMA.....a...		
00000040	00 09 70 48 59 73 00 00 0e c3 00 00 0e c3 01 c7	..pHYs.....		
00000050	6f a8 64 00 00 06 36 49 44 41 54 78 5e ed 9c bd	o.d...6IDATx^...		
00000060	92 b2 4a 10 86 df ef dc 08 1a 58 4e c0 3d 4c 99	..J.....XN.=L.		
00000070	68 b2 19 5c 00 91 46 46 26 54 11 53 65 62 b4 11	h..\..FF&T.Seb..		
00000080	46 5c 00 64 26 9a 58 dc 03 01 5b 06 ca 95 9c 13	F\d&.X...[.....		
00000090	00 3a fc 83 ca ee 77 76 fa a9 22 61 84 e9 19 e6	wv.. "a....		

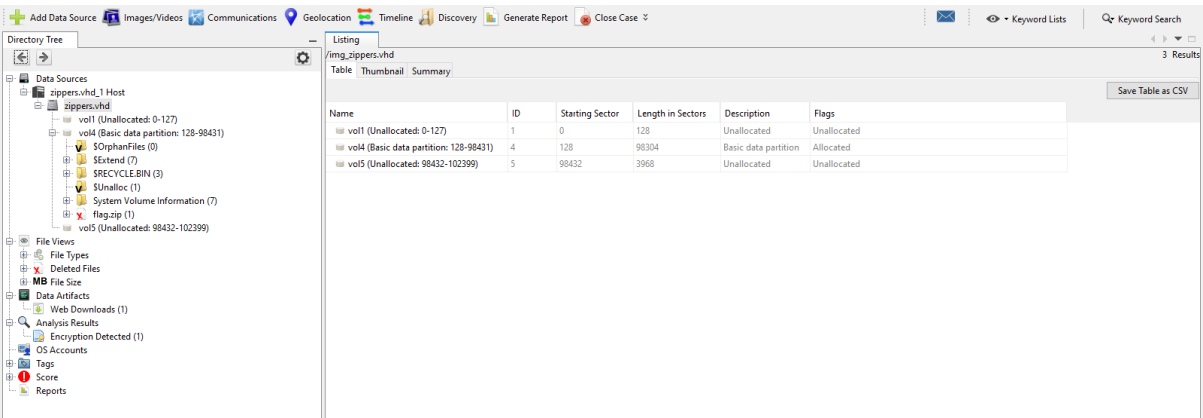
Porém, os 6 primeiros hexadecimais do arquivo estavam muito suspeitos: “de ad be ef ca fe”. Sem contar que o trecho “IHDR” na parte de texto parecia indicar algo relacionado a pngs, como comentado em aula. Dessa forma, utilizei o site <https://HexEd.it> para editar os magic bytes do arquivo, trocando esses 6 primeiros hexadecimais pelos 6 originais do png. Feito isso, acessei a imagem, e ela finalmente pôde ser vista, contendo a flag.



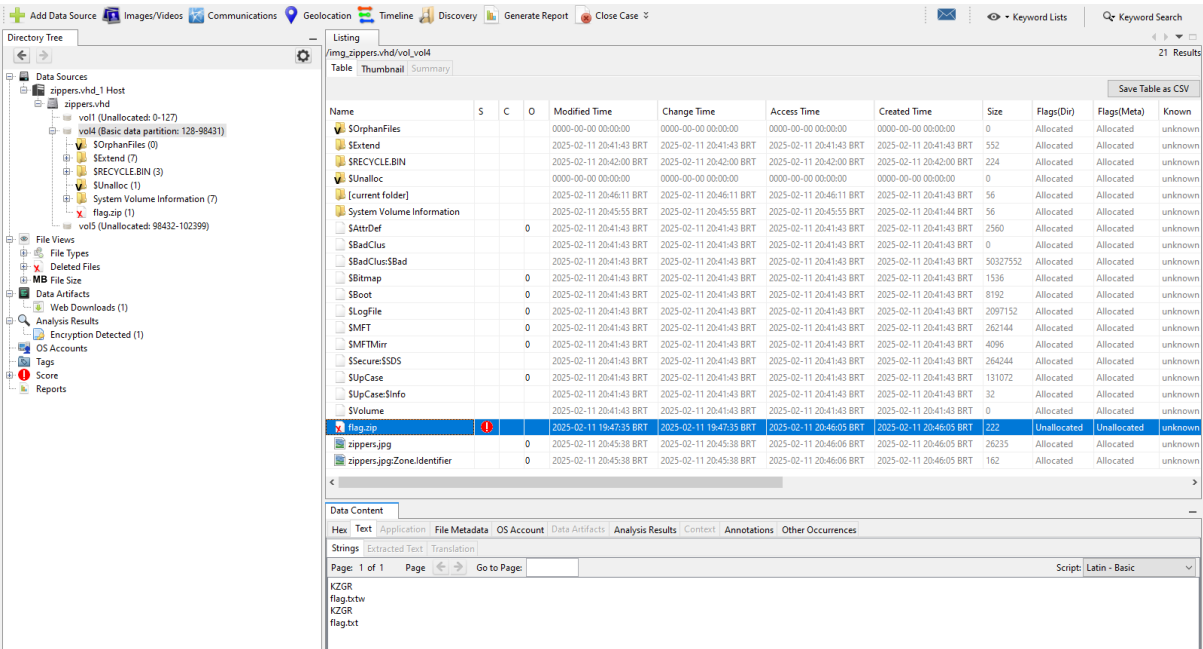
- zippers

Como o chall está relacionado à análise de disco de memória, baixei o FTK Imager em minha máquina virtual de Windows. Com o programa baixado, acessei a imagem de disco “zippers” e dei uma olhada em sua memória e seus caracteres associados. Não chegando a nenhum lugar, resolvi trocar de programa de análise.

Em minha segunda tentativa, baixei o Autopsy no Windows. Com esse programa instalado, criei um novo projeto, configurei suas características e adicionei o arquivo da imagem do disco. Feito isso, o projeto estava pronto para ser analisado:



Fazendo a análise do disco, foi possível encontrar algumas coisas interessantes, tal como a imagem citada na descrição do chall, que possui apenas zippers abertos e fechados, e um arquivo chamado “flag.zip” que teria sido deletado pelo usuário. Utilizando ASCII Strings para analisar seu conteúdo, é possível verificar que apresentava pelo menos um arquivo chamado “flag.txt”, e uma palavra-chave associada a KZGR. Vale notar que para fazer unzip dessa pasta no autopsy, é necessária uma senha.



Para investigar mais a respeito sobre esse arquivo zip e sua senha, extrai a pasta e a salvei em meu computador. Como observado anteriormente, o arquivo “flag.txt” está protegido por uma senha. Testagem de senhas para pastas zips não costuma apresentar limite de tentativas, dessa forma, uma forma de tentar acessar o arquivo é por meio de uma wordlist que contenha várias possíveis senhas para testagem por força bruta. Na descrição do chall, foi passado uma wordlist reduzida, assim, resolvi usar ela.

Para utilização da wordlist na testagem de senhas nessa análise do zip, baixei o programa “John the Ripper” (chamarei de JtR). No terminal do Windows, dentro do diretório desse programa, primeiramente criei um arquivo que contém as hashes que o JtR poderá processar no zip da flag, em seguida executei o comando que quebra a senha do txt a partir de uma dada wordlist. Com isso, consegui a senha do zip: “pictures”.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [versão 10.0.19045.3803]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\guilh>cd C:\Users\guilh\OneDrive\Área de Trabalho\john-1.9.0-jumbo-1-win64\run
C:\Users\guilh\OneDrive\Área de Trabalho\john-1.9.0-jumbo-1-win64\run>zip2john C:\Users\guilh\Downloads\flag.zip > hash_zip.txt
ver 2.0 flag.zip/flag.txt PKZIP Encr: cmplen=72, decmplen=60, crc=D58F5247

C:\Users\guilh\OneDrive\Área de Trabalho\john-1.9.0-jumbo-1-win64\run>john --wordlist=C:\Users\guilh\Downloads\reduced_wordlist.txt hash_zip.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
pictures (flag.zip/flag.txt)
lg 0:00:00:00 DONE (2025-03-05 01:18) 71.42g/s 75428p/s 75428c/s 75428C/s *@8502$.maiiumitha20
Use the "--show" option to display all of the cracked passwords reliably
Session completed

C:\Users\guilh\OneDrive\Área de Trabalho\john-1.9.0-jumbo-1-win64\run>john --wordlist=C:\Users\guilh\Downloads\reduced_wordlist.txt hash_zip.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
No password hashes left to crack (see FAQ)

C:\Users\guilh\OneDrive\Área de Trabalho\john-1.9.0-jumbo-1-win64\run>john --show hash_zip.txt
flag.zip/flag.txt:pictures:flag.txt:flag.zip::C:\Users\guilh\Downloads\flag.zip

1 password hash cracked, 0 left

C:\Users\guilh\OneDrive\Área de Trabalho\john-1.9.0-jumbo-1-win64\run>
```

Por fim, coloquei a senha no acesso ao arquivo “flag.txt” dentro do zip, e consegui a flag: CTF-BR{v0u_35c0nd3r_70d05_05_m3u5_cr1m35_3m_42qu1v05_z1p_:D}